

online-kitchen Technical Documentation

1. Project Overview

online-kitchen (also referred to as Digital Kitchen) is a backend food ordering system built with a robust, scalable architecture.

2. Tech Stack

- Runtime: Node.js
- Framework: Express.js
- Database: MongoDB with Mongoose ODM
- Validation: Zod for request schema validation
- Security: JWT for authentication, Bcrypt for password hashing

3. System Architecture

The project follows a Service-Oriented Architecture (SOA) with a focus on clean separation of concerns:

- Routes: Define endpoints and apply middleware.
- Controllers: Thin layer handling request/response logic.
- Services: Contain the core business logic and interact with models.
- Models: Mongoose schemas defining the data structure.
- Utils: Reusable utility functions and custom error classes.

Interaction Flow

'Client -> Route -> Middleware (Auth/Validation) -> Controller -> Service -> Model -> Database'

4. Core Data Models

User ('User')

- 'email' / 'phone': Unique identifiers (one is required).
- 'password': Hashed credentials.
- 'role': ["user", "admin"].

Food ('Food')

- 'name', 'description', 'price'.
- 'category', 'image'.
- 'isAvailable': Boolean flag.

Cart ('Cart')

- 'user': Reference to the User.
- 'items': Array of objects containing 'food' reference and 'quantity'.

Order ('Order')

- 'user', 'items' (snapshot of name/price at order time).
- 'totalAmount', 'status' (Pending, Preparing, etc.), 'deliveryAddress'.

5. API Endpoints Overview

Authentication

- 'POST /api/v1/auth/register': Register a new user.
- 'POST /api/v1/auth/login': Authenticate and receive a token.

Food

- 'GET /api/v1/food': List all available food items.
- 'POST /api/v1/food': Add new food (Admin only).

Cart

- 'GET /api/v1/cart': View current user's cart.
- 'PATCH /api/v1/cart': Add/Update items in the cart.

Orders

- 'POST /api/v1/orders': Place an order from the cart.
- 'GET /api/v1/orders': Get user order history.

6. Middleware & Security

- Authentication: 'auth.middleware.js' verifies JWT and attaches user to the request.
- Authorization: 'authorize.middleware.js' restricts access based on user roles (e.g., admin).
- Validation: 'validate.js' uses Zod schemas to ensure incoming data is correct before reaching controllers.
- Logging: 'logger.middleware.js' logs incoming requests for monitoring.

7. Error Handling

The application uses a centralized error-handling strategy:

- 'AppError': A custom class inheriting from 'Error' to handle operational errors with status codes.
- 'asyncHandler': (Used in routes) Wraps asynchronous functions to catch errors and pass them to the global handler.
- 'error.middleware.js': The final middleware that sends a formatted JSON response to the client.

8. Environment Setup & Running

Prerequisites

- Node.js (v16+ recommended)
- MongoDB account or local instance

Installation

1. Clone the repository.
2. Install dependencies:

```
""bash
npm install
""
```

Configuration

Create a '.env' file in the root directory and add the following variables:

```
PORT=5000
MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=your_long_random_string
JWT_EXPIRES_IN=7d
```

Running the Project

- Production Mode:

```
""bash
npm start
""
```

- Development Mode (with nodemon):

```
""bash
npm run dev
""
```

9. Development Workflow

- Validation: All new endpoints should have corresponding Zod validators in 'src/validators'.
- Logic: Business logic must remain in 'src/services', not in controllers.
- Testing: (Planned) Unit and integration tests to ensure system stability.